

Tarea 2: OpenCL y CUDA

Computación en GPU

Integrante: Matías Saavedra
Profesora: Nancy Hitschfeld K.
Auxiliar: Gustavo Medel
Ayudante: Catalina Gajardo

Fecha de entrega: 10 de junio de 2026
Santiago de Chile

Resumen

Este informe describe el desarrollo y la evaluación de implementaciones del Juego de la Vida de Conway en CPU y GPU, con el fin de medir las células evaluadas por segundo y el *speed-up* de la GPU respecto de la CPU. Se utiliza una variación de las reglas estándar en la que una célula muerta también nace si tiene exactamente 6 vecinos vivos (la regla *HighLife*, **B36/S23**). El núcleo del proyecto es agnóstico al backend: una grilla plana común, una única regla compartida y dos interfaces intercambiables (motor de simulación y *renderer*). La implementación de CUDA se compara bit a bit contra la CPU como oráculo. Se comparan dos variaciones de configuración del kernel (tamaño de bloque y uso de memoria compartida) y se analiza a partir de qué tamaño de cuadrícula $N \times M$ conviene usar la GPU. La GPU alcanza hasta $\approx 200\times$ sobre la mejor CPU paralela; el perfilamiento revela que el kernel está limitado por la latencia de su tubería de memoria en chip —no por el ancho de banda de DRAM, que queda al $\approx 20\%$ — y que la memoria compartida resulta contraproducente para este *stencil* de 1 byte.

Índice de Contenidos

1. Introducción	1
1.1. Resultados esperados	1
2. Implementación	1
2.1. Arquitectura general	1
2.2. Implementación en CPU (secuencial y paralela)	3
2.3. Implementación paralela en CUDA	4
2.4. Implementación paralela en OpenCL	7
2.5. Variaciones de configuración	7
2.6. Verificación de correctitud	8
2.7. Metodología de medición	8
2.8. Ejecutables y flujo de uso	8
3. Cotas teóricas de rendimiento	9
3.1. Modelo <i>roofline</i>	9
3.2. Hardware del equipo de pruebas	10
3.3. Cota de la GPU: DRAM, caché L2 y latencia	10
3.4. Cota de la CPU: memoria contra cómputo	11
3.5. Transferencias PCIe y el <i>ping-pong</i> en el <i>device</i>	12
3.6. Cota del <i>speed-up</i>	12
3.7. Validación del modelo	12
4. Resultados	13
4.1. Configuración experimental	13
4.2. CPU vs GPU por tamaño de cuadrícula	14
4.3. Resultados de las opciones elegidas	16
5. Análisis de resultados	17
5.1. Speed-up	17
5.2. Umbral de conveniencia GPU	17
5.3. Perfilamiento (Nsight / Chrono)	18
6. Conclusiones	19

Índice de Figuras

1.	Diagrama de clases del proyecto. La Application posee un ISimEngine y un IRenderer (las dos estrategias). CpuEngine , CudaEngine y OpenCLEngine realizan ISimEngine . Las flechas punteadas son dependencias: CpuEngine y CudaEngine usan LifeRules::nextState directamente, mientras que OpenCLEngine replica esa misma regla en kernel.cl (leído en tiempo de ejecución).	2
2.	Los tres espacios de coordenadas y las dos funciones que los conectan: los índices de hilo dan (row , col), y offset = row*cols + col aplanan la celda a su posición en el arreglo.	5
3.	Un bloque corresponde a un rectángulo en el tablero (arriba) pero a dos tramos <i>no contiguos</i> en memoria (abajo), separados por cols - bx . Ejemplo a escala reducida (cols=8 , bloque 4×2 , warp = 4); el código real usa un warp de 32.	6
4.	Vecindario de 9 puntos en <i>offsets</i> de memoria: horizontales idx ± 1 , verticales idx ± cols . El kernel los calcula como nr*cols + nc .	6
5.	Rendimiento (Mcélulas/s, ejes log-log) según el tamaño de grilla <i>N</i> , mejor de las repeticiones.	14
6.	Rendimiento de la CPU (Mcélulas/s) según el número de hilos; la recta punteada es el escalamiento lineal ideal.	15
7.	CUDA: rendimiento (Gcélulas/s) por tamaño de bloque, memoria global (sólido) frente a compartida (discontinuo), para varias grillas.	16
8.	Izquierda: <i>speed-up</i> de la GPU frente a la CPU secuencial y a la mejor paralela. Derecha: eficiencia wall/kernel (peso del <i>overhead</i> de lanzamiento); la banda señala la rodilla $N \approx 1024\text{--}2048$.	17

Índice de Tablas

1.	Especificaciones del equipo de pruebas que fijan las cotas.	10
2.	Cotas <i>roofline</i> (Ecuación 3) por backend y modelo de tráfico <i>q</i> .	11
3.	Cota de DRAM contra medición (mejor configuración). Ninguno satura su DRAM: a la GPU la limita la tubería L2/latencia (~65 % SOL); a la CPU, el cómputo. 1 Gcélula/s = 10^3 Mcélulas/s.	13
4.	Rendimiento por backend y <i>speed-up</i> de la GPU frente a la mejor CPU paralela (mejor de las repeticiones). En $N=16384$ la CPU secuencial se omitió por costo (su rendimiento es plano, ≈ 26 Mcélulas/s).	15
5.	Mejor configuración global frente a compartida (Gcélulas/s; el mejor tamaño de bloque se indica entre paréntesis).	16
6.	Nsight Compute: kernel global <i>vs.</i> compartido (bloque 128, $N = 4096$).	19

Índice de Códigos

1.	include/gol/LifeRules.hpp — la regla variante, compartida por CPU y CUDA.	2
2.	src/engines/cpu/CpuEngine.cpp — un paso (mismo código para secuencial y paralelo).	3
3.	src/engines/cuda/kernel.cu — kernel de memoria global (un hilo por celda). .	4
4.	src/engines/cuda/kernel.cu — mapeo de bloque y elección de kernel.	4
5.	src/engines/cuda/CudaEngine.cu — un paso con cronometraje por eventos. . .	7

1. Introducción

El Juego de la Vida de Conway es un autómata celular bidimensional en el que cada celda de una grilla $N \times M$ se encuentra viva o muerta y evoluciona en pasos discretos (generaciones) según el estado de sus ocho vecinos. En esta tarea se considera la siguiente *variación* de las reglas estándar:

- Una célula nace en un espacio muerto si tiene exactamente 3 vecinos vivos.
- Una célula sobrevive a la próxima generación si tiene 2 o 3 vecinos vivos.
- Una célula también nace en un espacio muerto si tiene exactamente 6 vecinos vivos.

El objetivo es implementar el algoritmo en CPU (secuencial), CUDA y OpenCL, medir su rendimiento en *células evaluadas por segundo*, comparar dos técnicas de optimización del kernel y analizar el *speed-up* respecto de la versión secuencial, determinando desde qué tamaño de grilla conviene usar la GPU.

1.1. Resultados esperados

El problema es un *stencil* de 9 puntos sobre celdas de 1 byte: muy poco cálculo por byte leído, es decir, limitado por ancho de banda de memoria (las cotas teóricas se derivan en la Sección 3). Se espera, en consecuencia:

- Que la GPU supere ampliamente a la CPU para grillas grandes, donde su mayor ancho de banda y paralelismo dominan frente al costo de transferencia *host* $\leftrightarrow$ *device*.
- Que para grillas pequeñas el costo de lanzamiento del kernel y de transferencia haga competitiva (o superior) a la CPU.
- Que el tamaño de bloque dé una curva cóncava que se estabiliza hacia 128–256 hilos, y que la memoria compartida aporte poco o nada, precisamente por tratarse de un *stencil* de 1 byte que ya reside en caché.

2. Implementación

2.1. Arquitectura general

El proyecto sigue el patrón *estrategia*: un núcleo agnóstico al backend más dos interfaces intercambiables. Lo único que difiere realmente entre backends — el cómputo de una generación — vive tras **ISimEngine**; la salida vive tras **IRenderer**; y la aplicación arma en tiempo de ejecución un motor y un *renderer* a partir de la configuración. La Figura 1 resume las clases y relaciones.

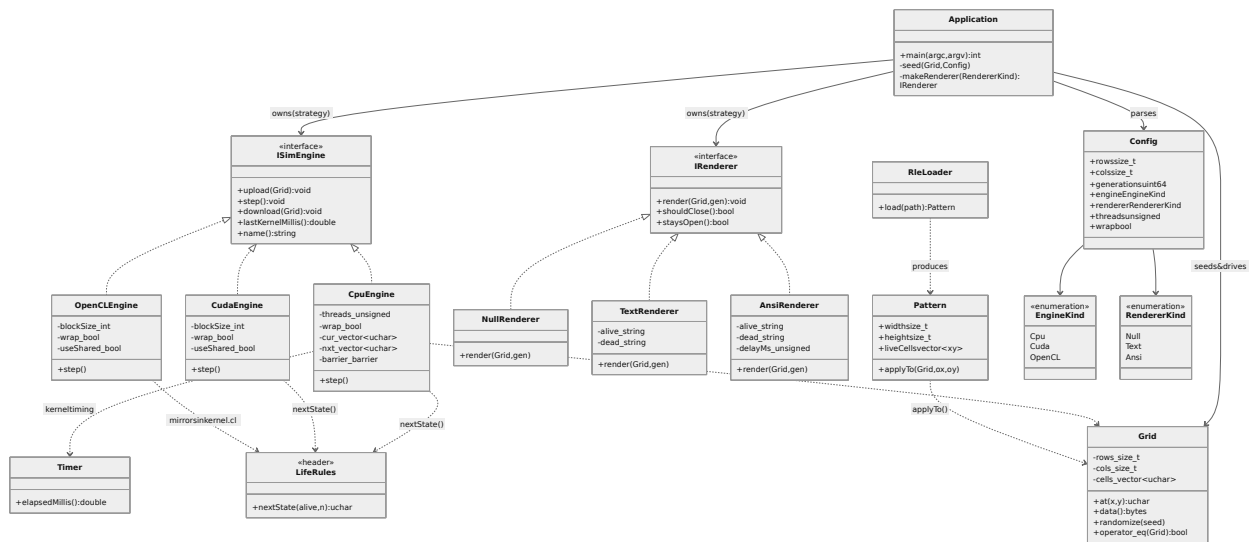


Figura 1: Diagrama de clases del proyecto. La **Application** posee un **ISimEngine** y un **IRenderer** (las dos estrategias). **CpuEngine**, **CudaEngine** y **OpenCLEngine** realizan **ISimEngine**. Las flechas punteadas son dependencias: **CpuEngine** y **CudaEngine** usan **LifeRules::nextState** directamente, mientras que **OpenCLEngine** replica esa misma regla en **kernel.cl** (leído en tiempo de ejecución).

Tres decisiones del núcleo hacen posible comparar los backends de forma justa:

Una grilla plana compartida.

Grid es un `std::vector<unsigned char>` contiguo, fila por fila, indexado $y \cdot \text{cols} + x$ (0 = muerta, 1 = viva). Es exactamente el formato de un buffer de *device*, de modo que subir/bajar la grilla es una copia sin traducción y los tres backends operan sobre los mismos bytes. Se acepta el costo de 1 byte por celda (en vez de empaquetar bits) para mantener el *stencil* como una simple lectura de arreglo, idéntica en CPU y GPU.

Una única regla, compilada para *host* y *device*.

La regla variante se escribe una sola vez en **LifeRules.hpp**. Bajo **nvcc** se califica `__host__ __device__`, de modo que la CPU y CUDA usan el mismo código fuente (Código 1); OpenCL no puede incluir encabezados de C++, por lo que replicará esta función en **kernel.cl**.

Código 1: `include/gol/LifeRules.hpp` — la regla variante, compartida por CPU y CUDA.

```

1 #ifdef __CUDACC__
2 #define GOL_FN __host__ __device__ inline
3 #else
4 #define GOL_FN inline
5 #endif
6
7 GOL_FN unsigned char nextState(unsigned char alive, int neighbors) {

```

```

8  if (alive)
9      return (neighbors == 2 || neighbors == 3) ? 1u : 0u; // sobrevive
10 return (neighbors == 3 || neighbors == 6) ? 1u : 0u; // nace (3 o 6)
11 }

```

Manejo de bordes idéntico.

Por defecto los bordes son acotados (un vecino fuera de rango cuenta como muerto); con `-wrap` se vuelven toroidales. Ambos modos deben coincidir bit a bit entre backends, y se verifican en las pruebas (Sección 2.6).

Contrato de doble buffer (*ping-pong*).

Cada motor mantiene dos buffers; `step()` lee el “actual”, escribe el “siguiente” y los intercambia, de modo que una llamada avanza exactamente una generación y nunca solapa entrada y salida. Las GPU hacen el intercambio en el *device*; la grilla del *host* solo se toca en `upload()/download()`.

2.2. Implementación en CPU (secuencial y paralela)

Una sola clase, `CpuEngine`, sirve tanto al *baseline* secuencial como a la versión paralela, porque el paso del Juego de la Vida no tiene dependencias intra-generación: cada celda lee solo el buffer actual y escribe el siguiente. Paralelizar es, entonces, repartir las *filas* entre hilos; la regla por celda es idéntica. El recuento de vecinos honra el modo de borde y suma los valores 0/1 de los vecinos; la regla compartida (`nextState`) decide el estado siguiente.

El reparto es por bandas de filas contiguas (buen comportamiento de caché y escrituras disjuntas). Con `threads == 1` no se crea ningún hilo ni barrera: el barrido secuencial es el caso degenerado de una sola partición. Con `threads >= 2` se usa un *pool* de hilos persistente sincronizado con una `std::barrier` de dos fases, de modo que el costo de crear hilos no contamina la métrica. El valor `threads == 0` se resuelve a `std::thread::hardware_concurrency()`.

Código 2: `src/engines/cpu/CpuEngine.cpp` — un paso (mismo código para secuencial y paralelo).

```

1 void CpuEngine::step() {
2     Timer t;
3     if (workers_.empty()) { // secuencial: un solo barrido, sin sincronización
4         computeRows(cur_.data(), nxt_.data(), 0, rows_);
5     } else { // paralelo: bandas de filas
6         src_ = cur_.data();
7         dst_ = nxt_.data();
8         barrier_>arrive_and_wait(); // libera a los workers sobre sus filas
9         auto [yBegin, yEnd] = rangeFor(0);
10        computeRows(src_, dst_, yBegin, yEnd); // el hilo main calcula la partición 0
11        barrier_>arrive_and_wait(); // espera a que todas las particiones terminen
12    }
13    std::swap(cur_, nxt_); // ping-pong
14    lastMs_ = t.elapsedMillis(); // cronometraje con std::chrono

```


15 }

Como secuencial y paralelo ejecutan la misma función de cómputo (`computeRows`) y solo difieren en el rango de filas, producen tableros idénticos; esto se comprueba en las pruebas (Sección 2.6).

2.3. Implementación paralela en CUDA

`CudaEngine` usa un hilo por celda sobre una grilla 2D de bloques. Posee dos buffers de *device* y realiza el *ping-pong* en el *device*; la grilla del *host* solo se toca en **upload** (copia H2D) y **download** (D2H). El encabezado del motor no incluye tipos de CUDA (los manejadores se guardan como punteros opacos), de modo que `main.cpp` compila con el compilador anfitrión y solo los `.cu` pasan por `nvcc`.

Hay dos kernels seleccionables en tiempo de ejecución. El de memoria global lee los ocho vecinos directamente; reutiliza `nextState` de `LifeRules.hpp` y replica el manejo de bordes de la CPU (Código 3).

Código 3: `src/engines/cuda/kernel.cu` — kernel de memoria global (un hilo por celda).

```

1  __global__ void lifeGlobal(const unsigned char* src, unsigned char* dst,
2                          int rows, int cols, bool wrap) {
3      const int col = blockIdx.x * blockDim.x + threadIdx.x;
4      const int row = blockIdx.y * blockDim.y + threadIdx.y;
5      if (col >= cols || row >= rows) return;
6      int n = 0;
7      for (int dy = -1; dy <= 1; ++dy)
8          for (int dx = -1; dx <= 1; ++dx) {
9              if (dx == 0 && dy == 0) continue;
10             int nc = col + dx, nr = row + dy;
11             if (wrap) { nc = (nc + cols) % cols; nr = (nr + rows) % rows; }
12             else if (nc < 0 || nr < 0 || nc >= cols || nr >= rows) continue; // borde: muerto
13             n += src[(size_t)nr * cols + nc];
14         }
15     const size_t idx = (size_t)row * cols + col;
16     dst[idx] = nextState(src[idx], n); // misma regla que la CPU
17 }
```

El lanzador mapea el tamaño de bloque sobre un *tile* 2D de ancho fijo 32 y elige el kernel. El ancho 32 alinea los accesos: hilos consecutivos de un *warp* leen celdas (de 1 byte) consecutivas de una fila, logrando lecturas coalescentes, lo correcto para un *stencil* limitado por memoria.

Código 4: `src/engines/cuda/kernel.cu` — mapeo de bloque y elección de kernel.

```

1  const int bx = 32; // filas de 32 -> lecturas coalescentes
```

```

2 const int by = (blockSize + bx - 1) / bx; // 32/64/128/256 -> by = 1/2/4/8
3 dim3 block(bx, by);
4 dim3 grid((cols + bx - 1) / bx, (rows + by - 1) / by);
5 if (useShared)
6     lifeShared<<<grid, block, (bx + 2) * (by + 2)>>>(src, dst, rows, cols, wrap);
7 else
8     lifeGlobal<<<grid, block>>>(src, dst, rows, cols, wrap);

```

Para entender ese mapeo conviene separar **tres espacios de coordenadas** (Figura 2): el *espacio de ejecución* (cómo se organizan los hilos: `blockIdx`, `threadIdx`, en 2D), el *espacio del tablero* (la grilla lógica `rows × cols`, indexada por `(row, col)`) y el *espacio de memoria* (el arreglo plano 1D en orden *row-major*, indexado por un único `offset`). El arreglo es siempre 1D; la indexación 2D de los hilos solo sirve para asignar un hilo por celda y para abaratar el cálculo de vecinos. Dos funciones los conectan: `col = blockIdx.x*32 + threadIdx.x`, `row = blockIdx.y*by + threadIdx.y`, y al acceder a memoria se aplanan con `offset = row*cols + col` (la expresión `nr*cols + nc` del Código 3).

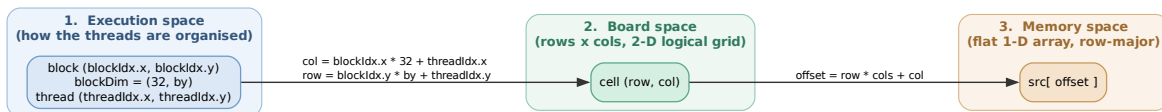


Figura 2: Los tres espacios de coordenadas y las dos funciones que los conectan: los índices de hilo dan `(row, col)`, y `offset = row*cols + col` aplanan la celda a su posición en el arreglo.

Una consecuencia clave es que un bloque de $32 \times by$ hilos es un **rectángulo en el tablero**, pero *no* un tramo contiguo en memoria (Figura 3): cada una de sus `by` filas es un tramo de 32 bytes contiguos, y filas consecutivas quedan separadas por `cols` celdas. Un *warp* (32 hilos con igual `threadIdx.y`) es justamente uno de esos tramos contiguos —por eso sus lecturas son coalescentes y por eso se fija `bx = 32`—. La celda “de abajo” está a `+cols`, no a `+bx`.

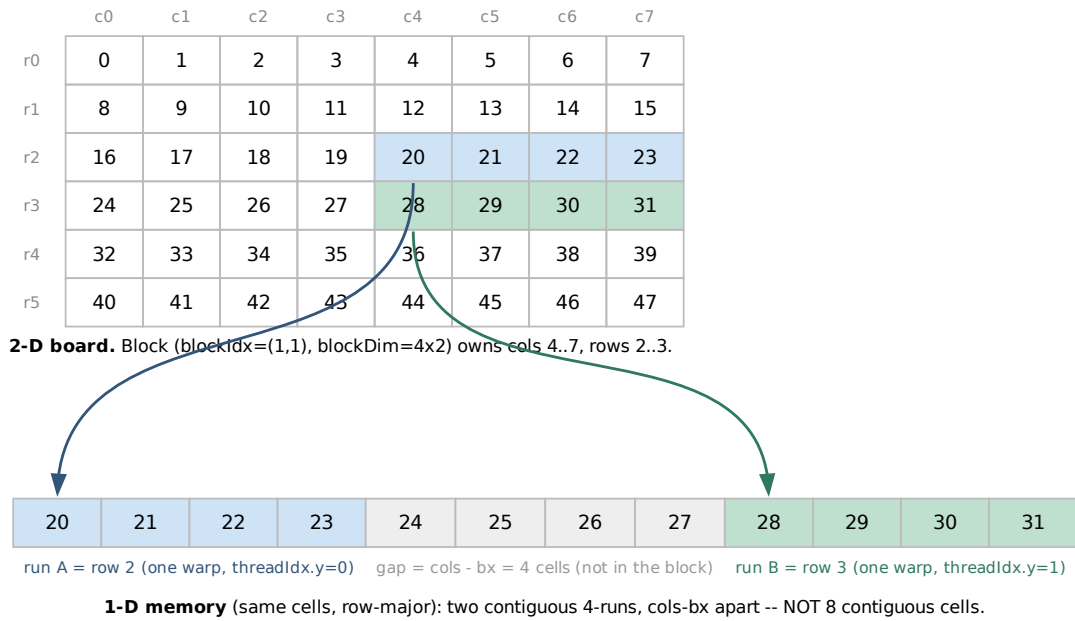


Figura 3: Un bloque corresponde a un rectángulo en el tablero (arriba) pero a dos tramos *no contiguos* en memoria (abajo), separados por $\text{cols} - \text{bx}$. Ejemplo a escala reducida ($\text{cols}=8$, bloque 4×2 , $\text{warp} = 4$); el código real usa un warp de 32.

Por la misma razón, los nueve vecinos del *stencil* tampoco son contiguos: el kernel reconstruye cada uno aritméticamente (Figura 4). Los vecinos horizontales están a $\text{idx} \pm 1$ y los verticales a $\text{idx} \pm \text{cols}$ (una fila completa de distancia), tal como hace $\text{nr} * \text{cols} + \text{nc}$ en el Código 3.

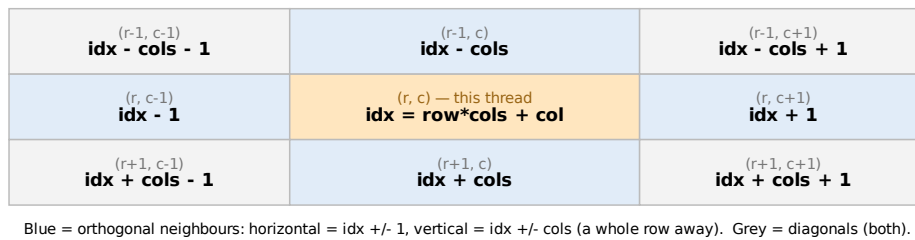


Figura 4: Vecindario de 9 puntos en *offsets* de memoria: horizontales $\text{idx} \pm 1$, verticales $\text{idx} \pm \text{cols}$. El kernel los calcula como $\text{nr} * \text{cols} + \text{nc}$.

El kernel de memoria compartida (`-shared`) primero carga cooperativamente un *tile* de $(bx+2) \times (by+2)$ celdas (las del bloque más un halo de una celda) en memoria compartida —aplicando el mismo manejo de borde durante la carga del halo— y luego lee los vecinos desde el *tile*. Ambos kernels producen resultados idénticos al oráculo de CPU.

El cómputo se cronometra con `cudaEvent_t` alrededor del lanzamiento, de modo que `lastKernelMillis()` reporta solo el tiempo del kernel, excluyendo las transferencias; es el análogo en GPU del cronometraje con `std::chrono` de la CPU.

Código 5: `src/engines/cuda/CudaEngine.cu` — un paso con cronometraje por eventos.

```

1 void CudaEngine::step() {
2     cudaEventRecord(start);
3     launchLifeStep(dCur_, dNxt_, rows_, cols_, wrap_, blockSize_, useShared_);
4     cudaEventRecord(stop);
5     cudaEventSynchronize(stop);
6     cudaEventElapsedTime(&ms, start, stop); // solo el kernel, sin H2D/D2H
7     lastMs_ = ms;
8     std::swap(dCur_, dNxt_);               // ping-pong en el device
9 }
```

El *target* de compilación es `sm_75` (la GPU de pruebas, una GTX 1660 Ti), con `-lineinfo` para que Nsight Compute pueda atribuir tiempo a líneas de código fuente.

2.4. Implementación paralela en OpenCL

Pendiente. El motor de OpenCL (`OpenCLEngine`) aún no está implementado. Seguirá el mismo diseño que CUDA: dos buffers de *device*, *ping-pong* en el *device*, los mismos dos kernels (global y con memoria local) y los mismos botones `-block/-shared`. La regla y el manejo de borde se replicarán en un `kernel.cl` en C plano, leído en tiempo de ejecución y mantenido diff-eable contra `LifeRules.hpp`. El cronometraje del kernel usará eventos de la cola de comandos con `CL_QUEUE_PROFILING_ENABLE`. Se verificará bit a bit contra el oráculo de CPU, igual que CUDA.

2.5. Variaciones de configuración

De las tres opciones del enunciado se eligieron dos:

1. **Tamaño de bloque** $\in \{32, 64, 128, 256\}$ (botón `-block`).
2. **Memoria local por bloque** *vs.* sin memoria local (botón `-shared`).

No se eligió la opción de arreglos 2D *vs.* mapeo a 1D porque el diseño fija una grilla plana 1D compartida por todos los backends; usar arreglos 2D iría en contra de esa decisión arquitectónica. Ambas variaciones son seleccionables en tiempo de ejecución, de modo que un único binario ejecuta toda la matriz de experimentos.

2.6. Verificación de correctitud

CpuEngine es el oráculo de referencia. Las pruebas (GoogleTest) verifican, en orden de dependencia:

- **Regla variante en CPU:** un bloque permanece estático, un *blinker* oscila con período 2 y una célula muerta con exactamente 6 vecinos nace (el caso que define la variante).
- **Equivalencia secuencial vs. paralela:** misma semilla y mismas generaciones producen un tablero idéntico bit a bit para distinto número de hilos y ambos modos de borde, incluso en grillas no divisibles.
- **Equivalencia entre backends (CUDA):** `cuda_equivalence_test.cpp` siembra CUDA y CPU igual y exige igualdad bit a bit contra el oráculo para los tamaños de bloque $\{32, 64, 128, 256\}$, kernels global/compartido y ambos modos de borde. OpenCL seguirá el mismo patrón.

Adicionalmente, el runner **bgolly** de Golly (grilla infinita) simula exactamente **B36/S23** y sirve como oráculo externo independiente.

2.7. Metodología de medición

La métrica principal es *células evaluadas por segundo*:

$$\text{Mcélulas/s} = \frac{N \cdot M \cdot G}{t_{\text{transcurrido}} \times 10^6}, \quad (1)$$

con G el número de generaciones. El cronometraje es uniforme y en programa (no desde un *profiler*), expuesto por `ISimEngine::lastKernelMillis()`: `std::chrono` en CPU, `cudaEvent_t` en CUDA y eventos de cola en OpenCL. Se reporta el tiempo de kernel por separado del de transferencias, y se mide siempre con el `NullRenderer` para que el costo de salida y de transferencia no contamine la métrica. El *speed-up* es

$$S(N) = \frac{T_{\text{CPU}}(N)}{T_{\text{GPU}}(N)}. \quad (2)$$

Para los barridos, el binario ofrece un modo CSV: `-csv` imprime una fila por corrida (columnas `backend`, `rows`, `cols`, `generations`, `threads`, `block`, `shared`, `wrap`, `kernel_ms`, `wall_ms`, `mcells_kernel`, `mcells_wall`) y `-csv-header` imprime solo el encabezado; así un *script* escribe el encabezado una vez y agrega una fila por configuración, lista para Pandas.

2.8. Ejecutables y flujo de uso

El proyecto compila un único ejecutable `gol` que arma en tiempo de ejecución el motor (`-engine cpu|cuda|opencl`) y el *renderer* (`-renderer null|text|ansi`) según la línea de comandos, siembra la grilla (con una semilla aleatoria determinista o un patrón RLE) y

ejecuta el bucle de generaciones. Bajo `NullRenderer` el bucle es solo `step()`, sin transferencias ni dibujado, de modo que la medición no se contamina. Los motores de GPU son opcionales en CMake (`-DBUILD_CUDA=ON`); el binario funciona en una máquina sin CUDA ni OpenCL.

3. Cotas teóricas de rendimiento

Antes de medir conviene fijar el techo: ¿cuál es el máximo de Mcélulas/s que el hardware permitiría en el mejor de los casos? El paso del Juego de la Vida es un *stencil* de 9 puntos sobre celdas de 1 byte: por cada celda se leen sus ocho vecinos, se hacen unas pocas sumas enteras y dos comparaciones, y se escribe un byte. La razón entre cálculo y tráfico de memoria (la *intensidad aritmética*) es minúscula, de modo que el factor limitante no es la capacidad de cómputo sino el **ancho de banda de memoria**. Esta sección deriva, desde el hardware del equipo de pruebas, una cota superior de rendimiento para cada backend y la cota de *speed-up* que de ellas se desprende; las mediciones (Secciones 4 y 5) se interpretan contra ellas.

3.1. Modelo *roofline*

Si una generación está limitada por memoria, su tiempo lo fija cuántos bytes deben viajar desde/hacia la DRAM, no cuántas operaciones se ejecutan. Sea B el ancho de banda sostenido de memoria (bytes/s) y q el tráfico de DRAM por celda y por generación (bytes/celda). El número de células evaluadas por segundo no puede superar

$$P_{\max} = \frac{B}{q}. \quad (3)$$

El valor de q depende del reúso de datos:

- **Reúso ideal** ($q = 2$): cada celda se trae de la DRAM una sola vez —sirve como vecina de sus ocho contiguas desde la caché— y se escribe una vez. Es el piso de tráfico: una lectura y una escritura por celda.
- **Sin caché** ($q \approx 10$): cada hilo *solicita* los nueve bytes de su vecindario 3×3 más la escritura. Es el tráfico que vería la DRAM si ninguna caché interviniera. En la práctica sí interviene: en la GPU la L2 sirve casi todas esas solicitudes y devuelve el tráfico real de DRAM a $q \approx 2$ (medido en la Sección 5.3).

La cota de cómputo aritmético queda muy por encima: la GTX 1660 Ti entrega $1536 \times 2 \times 2,1 \text{ GHz} \approx 6,5 \text{ TFLOP/s}$ que, repartidos entre las ~ 10 operaciones enteras por celda, darían cientos de Gcélulas/s. El problema es, pues, *memory-bound* —lo limita el sistema de memoria, no la ALU—, si bien el perfilamiento (Sección 5.3) precisa *qué* parte de ese sistema: no la DRAM, sino la tubería en chip (L2/LSU) y la latencia de las cargas.

3.2. Hardware del equipo de pruebas

La Tabla 1 resume los recursos relevantes. Los anchos de banda de memoria son los teóricos (de catálogo); la velocidad de la DRAM es la máxima soportada por el procesador (DDR4-2666, doble canal).

Tabla 1: Especificaciones del equipo de pruebas que fijan las cotas.

Recurso	Especificación	Magnitud derivada
CPU	Intel Core i7-9750H, 6C/12T	2,6–4,5 GHz
L1d / L1i	32 KiB / 32 KiB por núcleo	—
L2	256 KiB por núcleo (1,5 MiB)	—
L3	12 MiB compartida	—
DRAM	DDR4-2666, doble canal	$\approx 42,7$ GB/s
GPU	GTX 1660 Ti (Turing, <code>sm_75</code>)	1536 <i>cores</i>
VRAM	6 GiB GDDR6, bus 192 bits, 12 Gbit/s/pin	288 GB/s
Caché L2 GPU	1,5 MiB	—
PCIe	Gen3 $\times 16$	$\approx 15,8$ GB/s

3.3. Cota de la GPU: DRAM, caché L2 y latencia

La VRAM es GDDR6 sobre un bus de 192 bits a 12 Gbit/s por pin. Como GDDR6 transfiere en ambos flancos del reloj (DDR), el reloj de memoria que reporta la GPU (6001 MHz) se duplica a ~ 12 Gbit/s efectivos por pin, de donde

$$B_{\text{GPU}} = 12 \frac{\text{Gbit}}{\text{s} \cdot \text{pin}} \times \frac{192 \text{ bits}}{8} = 288 \text{ GB/s.} \quad (4)$$

El kernel global emite ~ 10 *solicitudes* de memoria por celda (nueve vecinos más la escritura), lo que sugeriría $q \approx 10$ y un techo de $\approx 28,8$ Gcélulas/s (fila “sin caché” de la Tabla 2). **Pero esa lectura es engañosa:** hay que separar las *solicitudes* —que la caché puede absorber— del *tráfico real a DRAM*. El perfilamiento con Nsight Compute (Sección 5.3) muestra que la L2 sirve el $\sim 88\%$ de esas solicitudes, de modo que la DRAM solo ve $q \approx 2$ bytes/celda y queda al $\approx 20\%$ de su ancho de banda. La cota de DRAM *pertinente* es por tanto la de $q = 2$, es decir $288/2 = 144$ Gcélulas/s, y el kernel alcanza $\approx 28\text{--}32$, o sea solo $\approx 20\%$ de ese techo. **La GPU no está limitada por el ancho de banda de DRAM** sino por la tubería de memoria en chip (L2/LSU) y la latencia de uso de las cargas: cómputo y tubería de memoria quedan co-limitados al $\sim 65\%$ de su máximo, con la mayoría de los ciclos detenidos esperando los datos de las cargas. El recurso que manda es L2, no GDDR6.

Tabla 2: Cotas *roofline* (Ecuación 3) por backend y modelo de tráfico q .

Backend	B	q (B/celda)	$P_{\max}$
GPU — DRAM con L2 ($q \approx 2$, medido)	288 GB/s	2	144 Gcélulas/s
GPU — sin caché ($q = 10$, hipotético)	288 GB/s	10	28,8 Gcélulas/s
CPU — DRAM ($q = 2$)	42,7 GB/s	2	21,3 Gcélulas/s

Esto explica con precisión por qué la **memoria compartida no puede ayudar** aquí. Su único beneficio sería reducir el tráfico de DRAM, pero la L2 ya lo dejó en $q \approx 2$: el perfilamiento confirma que la versión compartida mueve los mismos ≈ 2 bytes/celda a DRAM. A cambio *pierde* el reúso barato de L1 (la tasa de acierto cae del 67 % al 25 %, pues las cargas pasan ahora por la memoria compartida) y añade una pasada de carga cooperativa y la barrera `__syncthreads()`. El resultado es $\approx 1,9\times$ más lento pese a una ocupación mayor (Sección 4.3): se quita presión a un recurso (la DRAM) que no era el cuello de botella y se añade serialización.

3.4. Cota de la CPU: memoria contra cómputo

La DRAM del equipo (DDR4-2666 en doble canal) entrega

$$B_{\text{CPU}} = 2666 \frac{\text{MT}}{\text{s}} \times 8 \text{ B} \times 2 \text{ canales} \approx 42,7 \text{ GB/s}, \quad (5)$$

que con $q = 2$ daría una cota de 21,3 Gcélulas/s. Sin embargo, **esa cota no es la que limita a la CPU**, y la razón está en las cachés. El barrido procede fila por fila, y una fila solo necesita tres filas del buffer de entrada (la anterior, la actual y la siguiente). Para una grilla de 8192 columnas eso son

$$3 \times 8192 \times 1 \text{ byte} = 24 \text{ KiB} < 32 \text{ KiB (L1d)},$$

de modo que las tres filas caben en la caché L1 de datos de cada núcleo: tras la primera fila, las lecturas del vecindario se sirven desde L1 y la DRAM casi no se toca. La CPU no está limitada por ancho de banda sino por *cómputo*: cuántas instrucciones (cálculo de índices, ocho lecturas, sumas, el salto de la regla) puede retirar cada núcleo por ciclo.

La medición lo confirma: la versión de 12 hilos alcanza $\approx 0,16$ Gcélulas/s, apenas el 0,75 % del techo de DRAM. Repartido entre 12 hilos son ~ 12 Mcélulas/s por hilo; a un reloj sostenido de $\sim 3,5$ GHz eso son unos **300 ciclos por celda**, coherente con un *kernel* escalar, con un salto por celda y verificación de bordes, sin vectorizar. La consecuencia para el informe es clara: el techo realista de la CPU *tal como está implementada* es del orden de 0,16 Gcélulas/s, no los 21 del límite de memoria; solo una reescritura vectorizada (AVX2) y sin saltos se acercaría a ese límite.

3.5. Transferencias PCIe y el *ping-pong* en el *device*

El enlace PCIe Gen3 $\times 16$ ofrece $\approx 15,8$ GB/s teóricos (~ 12 GB/s efectivos). La métrica de esta tarea no lo incluye —se cronometra solo el kernel y se mide con el `NullRenderer`—, pero su cota justifica una decisión de diseño. Si se copiara la grilla entre *host* y *device* en cada generación (1 byte/celda, un sentido), el techo sería ~ 12 Gcélulas/s, por debajo de los ~ 28 que el kernel sostiene; copiar en ambos sentidos lo bajaría a ~ 6 . Por eso los motores de GPU mantienen los dos buffers en el *device* y hacen ahí el *ping-pong* (Sección 2.3): el bus PCIe se cruza solo una vez al subir la semilla y otra al bajar el resultado, nunca dentro del bucle de medición. (En reposo el enlace negocia Gen1 para ahorrar energía; sube a Gen3 bajo carga.)

3.6. Cota del *speed-up*

Si *ambos* backends estuvieran limitados por memoria con el mismo q , el *speed-up* máximo sería exactamente el cociente de anchos de banda:

$$S_{\max} = \frac{B_{\text{GPU}}}{B_{\text{CPU}}} = \frac{288}{42,7} \approx 6,8. \quad (6)$$

Este es el *speed-up* “justo” que predice el hardware. El valor medido (GPU contra 12 hilos) será mucho mayor —del orden de $200\times$ — precisamente porque la CPU **no** está en su techo de memoria: opera a $\sim 1/150$ de él (Sección 3.4). Es decir, un *speed-up* de $\sim 200\times$ no mide una ventaja de hardware de $200\times$. Conviene matizar: ni la CPU ni la GPU saturan su DRAM —la CPU está limitada por cómputo y la GPU por la latencia de su tubería de memoria (Sección 3.3)—. El $200\times$ refleja, entonces, el paralelismo de la GPU ocultando latencia frente a una CPU escalar; el cociente de anchos de banda ($\sim 6,8\times$) es solo una referencia de “qué pasaría si ambos saturaran su DRAM”. El informe reporta ambos para no confundir la ventaja de ingeniería con la de hardware puro.

3.7. Validación del modelo

La Tabla 3 contrasta las cotas con la medición y el perfilamiento (Sección 5.3). El hallazgo central corrige la intuición inicial: el kernel global **no está limitado por el ancho de banda de DRAM**. La L2 sirve el $\sim 88\%$ de las solicitudes, así que la DRAM solo ve $q \approx 2$ B/celda y opera al $\approx 20\%$ de su capacidad; el kernel alcanza $\approx 28\text{--}32$ Gcélulas/s, apenas el $\approx 20\%$ de la cota de DRAM de 144 Gcélulas/s, y su límite real es la tubería de memoria en chip y la latencia de las cargas ($\sim 65\%$ de SOL). La CPU, en cambio, se queda $\sim 130\times$ por debajo de su techo de DRAM —limitada por cómputo—.

Tabla 3: Cota de DRAM contra medición (mejor configuración). Ninguno satura su DRAM: a la GPU la limita la tubería L2/latencia ($\sim 65\%$ SOL); a la CPU, el cómputo. $1 \text{ Gcélula/s} = 10^3 \text{ Mcélulas/s}$.

Backend / config.	Cota DRAM	Medido	% cota
GPU global (DRAM, L2 da $q \approx 2$)	144 Gcél/s	25–32 (máx 32,1) Gcél/s	$\approx 20\%$
CPU, 12 hilos (DRAM, $q = 2$)	21,3 Gcél/s	$\approx 0,16 \text{ Gcél/s}$	0,75 %

En suma, el **máximo obtenible** en este proyecto queda así: la GPU rinde $\approx 25\text{--}32$ Gcélulas/s, no contra el techo de DRAM (144) sino contra el de su tubería de memoria y latencia. Hay, por tanto, margen —ocultar mejor la latencia con más paralelismo de instrucción (varias celdas por hilo) podría acercarla al techo de DRAM—; reducir el tráfico (empaquetar bits) ayudaría poco mientras la DRAM esté al 20 %. La CPU ronda $0,16$ Gcélulas/s, dominada por el cómputo, no por la memoria. El cociente de anchos de banda ($\sim 6,8\times$) es solo una referencia: ninguno de los dos satura su DRAM.

4. Resultados

4.1. Configuración experimental

Las mediciones se realizan en un equipo con procesador Intel Core i7-9750H (6 núcleos físicos / 12 hilos; L1d de 32 KiB por núcleo, L2 de 1,5 MiB y L3 de 12 MiB) y GPU NVIDIA GeForce GTX 1660 Ti (Turing, capacidad de cómputo 7.5; GDDR6 de 192 bits, 288 GB/s), bajo Linux con CUDA y compilación en C++23.

Los barridos se automatizan con `scripts/sweep.sh`, que invoca el binario en modo CSV y produce dos archivos en `results/`: uno para el barrido de optimizaciones de CUDA (bloque $\times$ memoria compartida) y otro para el escalamiento CPU/GPU según el tamaño de grilla; el análisis y las figuras se generan con el cuaderno `analysis/results.ipynb`. Cada configuración se ejecuta varias veces y se conserva el **mejor** tiempo (menor ruido de *turbo* y contención); todas las corridas usan el `NullRenderer` (sin dibujado ni transferencias en el lazo cronometrado), bordes acotados y la misma semilla.

Se miden grillas cuadradas $N \in \{64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384\}$. El número de generaciones se escala con N (muchas para grillas pequeñas, pocas para las grandes) de modo que cada corrida sea medible pero acotada. El motor de CPU se barre en $\{1, 2, 4, 6, 8, 10, 12\}$ hilos (1 = secuencial) y el de CUDA en bloques $\{32, 64, 128, 256, 512\}$ con y sin memoria compartida. Antes de los barridos se verificó que la GPU operara a plena velocidad (límite de potencia de 60 W, reloj de memoria 6001 MHz): un *cap* de potencia rebajaría el ancho de banda efectivo e invalidaría la comparación.

4.2. CPU vs GPU por tamaño de cuadrícula

La Figura 5 compara, en ejes log-log, los tres caminos según el tamaño de grilla. La CPU secuencial es prácticamente **plana** en ≈ 27 Mcélulas/s: como se anticipó en la Sección 3.4, está limitada por cómputo, no por memoria. La CPU paralela se estabiliza en ≈ 150 – 160 , y la GPU domina por **dos a tres órdenes de magnitud**, con su máximo (≈ 32 Gcélulas/s) en $N = 2048$; más allá decae levemente (efecto de reloj/SOL: el perfilamiento muestra que la tasa de acierto de L2 se mantiene en $\approx 88\%$ y la DRAM al $\approx 20\%$ también en $N = 8192$). La curva **CUDA (wall)** incluye el costo de lanzamiento por paso: se separa del **kernel** en grillas pequeñas (la GPU queda infrautilizada) y se le junta para $N \gtrsim 2048$.

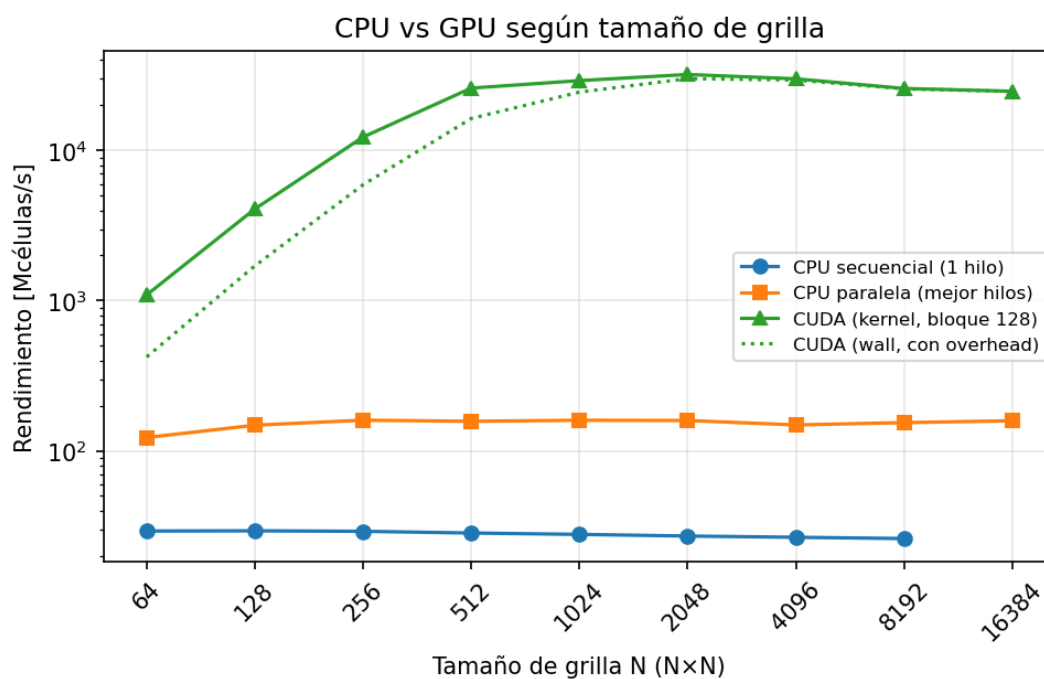


Figura 5: Rendimiento (Mcélulas/s, ejes log-log) según el tamaño de grilla N , mejor de las repeticiones.

Tabla 4: Rendimiento por backend y *speed-up* de la GPU frente a la mejor CPU paralela (mejor de las repeticiones). En $N=16384$ la CPU secuencial se omitió por costo (su rendimiento es plano, ≈ 26 Mcélulas/s).

N	CPU sec. [Mcél/s]	CPU par. [Mcél/s]	CUDA [Gcél/s]	S (vs par.)
256	29	160	12,3	77
1024	28	160	29,2	183
2048	27	159	32,1	202
4096	27	149	30,1	202
8192	26	154	25,9	168
16384	—	159	24,8	157

Escalamiento en CPU.

La Figura 6 muestra el rendimiento de la CPU según el número de hilos. El reparto por bandas de filas escala **casi linealmente hasta los 6 núcleos físicos** ($\approx 5,4\times$ con 6 hilos); el *hyperthreading* aporta poco (12 hilos $\approx 5,9\times$), coherente con un problema limitado por cómputo. El detalle notable es la **caída no monótona en 8 y 10 hilos**: con 8 hilos, 2 de los 6 núcleos ejecutan 2 hilos HT (cada uno $\sim 0,65\times$) y 4 ejecutan 1; como la `std::barrier` sincroniza cada generación, el paso espera al hilo más lento y los núcleos con HT se vuelven el cuello de botella. Con 12 hilos todos quedan igualmente cargados y el rendimiento se recupera. (No es térmico: el calentamiento haría caer también a 10 y 12 hilos.)

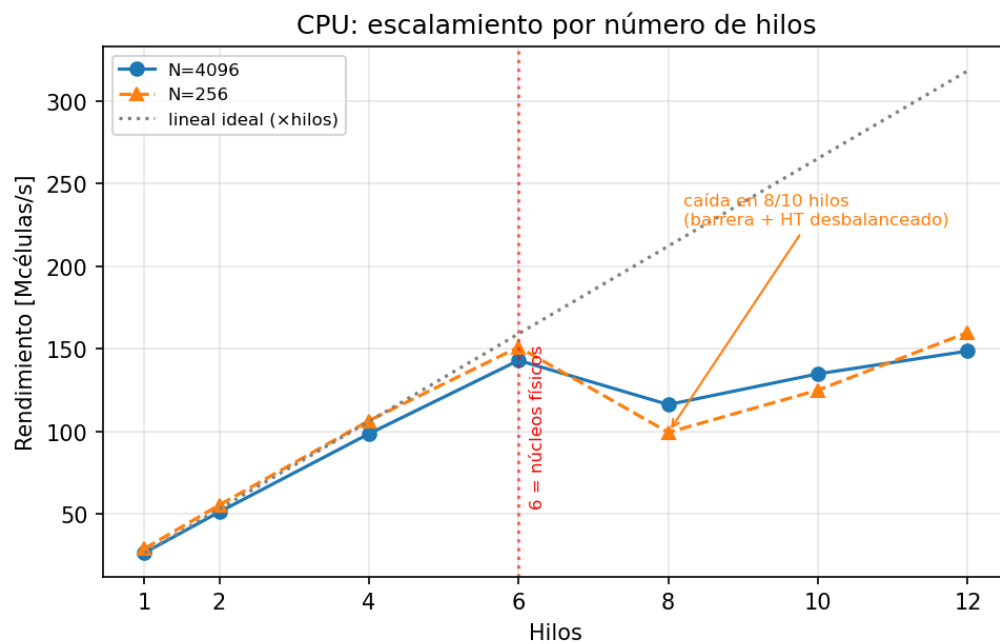


Figura 6: Rendimiento de la CPU (Mcélulas/s) según el número de hilos; la recta punteada es el escalamiento lineal ideal.

4.3. Resultados de las opciones elegidas

La Figura 7 reúne las dos optimizaciones. La curva por **tamaño de bloque** es cóncava: con 32 hilos hay muy pocos *warps* por SM para ocultar la latencia de memoria, y con 512 la ocupación cae; el óptimo está en **64–128** hilos. La **memoria compartida** (líneas discontinuas) queda *sistemáticamente por debajo* de la versión global —en torno a $0,55\times$ su rendimiento— confirmando la predicción del modelo (Sección 3.3): para un *stencil* de 1 byte la caché L2 ya sirve la reutilización, y la barrera, el halo y los accesos byte-a-byte a la SRAM cuestan más de lo que ahorran. La Tabla 5 resume el mejor bloque por N .

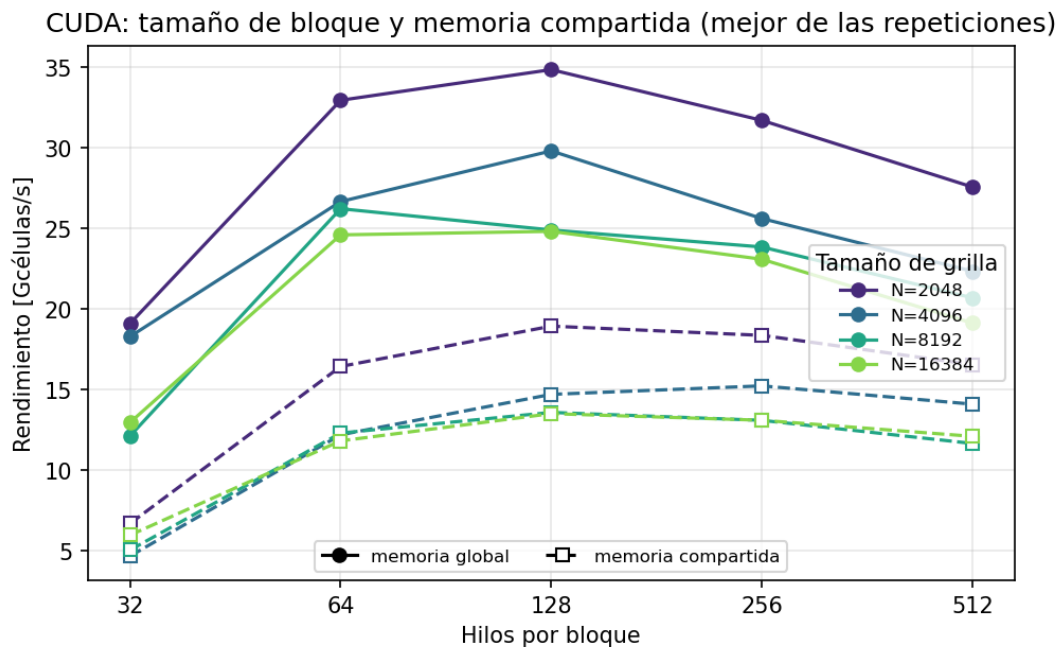


Figura 7: CUDA: rendimiento (Gcélulas/s) por tamaño de bloque, memoria global (sólido) frente a compartida (discontinuo), para varias grillas.

Tabla 5: Mejor configuración global frente a compartida (Gcélulas/s; el mejor tamaño de bloque se indica entre paréntesis).

N	global (mejor blq.)	compartida (mejor blq.)	global/compartida
1024	31,7 (64/128)	17,9 (128)	1,77
2048	34,9 (128)	18,9 (128)	1,85
4096	29,8 (128)	15,2 (256)	1,96
8192	26,2 (64)	13,6 (128)	1,93
16384	24,8 (128)	13,5 (128)	1,84

5. Análisis de resultados

5.1. Speed-up

El *speed-up* (Ecuación 2, panel izquierdo de la Figura 8) alcanza $\approx 1200\times$ frente a la versión secuencial y $\approx 200\times$ frente a la mejor CPU paralela, con su máximo en $N = 2048$ – 4096 . Conviene leer ambos números junto con la cota de la Sección 3.6: el cociente “limpio” de anchos de banda predice $\approx 6,8\times$, y el $\sim 200\times$ medido es mucho mayor porque la CPU base no está en su techo de memoria (opera a $\sim 1/150$ de él), no porque el hardware de la GPU sea $200\times$ superior. El $\sim 200\times$ mezcla, entonces, la ventaja real de ancho de banda de la GPU con la inmadurez de la CPU escalar de referencia.

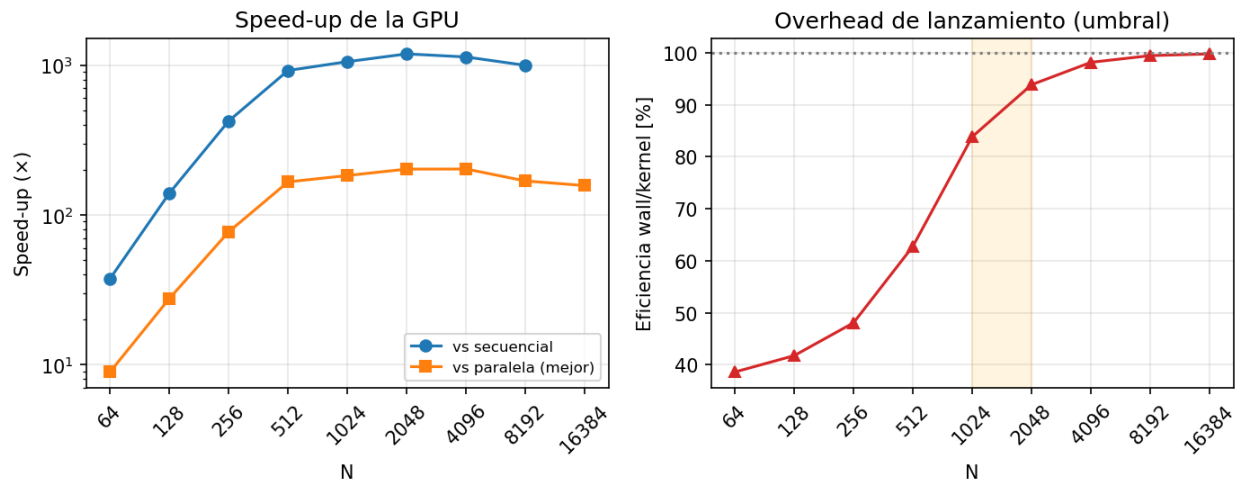


Figura 8: Izquierda: *speed-up* de la GPU frente a la CPU secuencial y a la mejor paralela. Derecha: eficiencia **wall/kernel** (peso del *overhead* de lanzamiento); la banda señala la rodilla $N \approx 1024$ – 2048 .

5.2. Umbral de conveniencia GPU

El panel derecho de la Figura 8 muestra la eficiencia **wall/kernel**, que mide el peso del *overhead* de lanzamiento por paso: sube de $\approx 40\%$ en $N = 64$ a $\approx 98\%$ en $N \gtrsim 2048$, de modo que la **rodilla** de conveniencia está en $N \approx 1024$ – 2048 . Por debajo, la GPU está infrautilizada (pocos bloques para llenar los 24 SM) y el lanzamiento domina; por encima, el kernel satura el bus.

Cabe una precisión sobre el umbral clásico “a partir de qué N conviene la GPU”. Con esta metodología —cronometraje de kernel/lazo con **NullRenderer**, que *excluye* las transferencias— la GPU supera a la CPU en *todo* el rango medido, incluso en $N = 64$: su único costo es el lanzamiento y la CPU base es débil. Un umbral por debajo del cual convenga la CPU solo aparece al contabilizar el costo PCIe de subir/bajar la grilla (Sección 3.5): transferir la grilla en cada generación acotaría la GPU a ~ 12 Gcélulas/s —en la práctica ~ 8 , medido en

la Sección 5.3—, y para grillas muy pequeñas el costo fijo de transferencia la haría perder frente a la CPU.

5.3. Perfilamiento (Nsight / Chrono)

Se perfiló la mejor solución CUDA (memoria global, bloque 128) con Nsight Systems (línea de tiempo) y Nsight Compute (contadores internos del kernel). Estas herramientas *no* producen los números de rendimiento —esos vienen del cronometraje en programa— sino que explican *por qué* el kernel rinde lo que rinde.

Nsight Systems (nsys).

Sobre 200 generaciones en 8192², cada lanzamiento de `lifeGlobal` dura $\approx 2,3\text{--}2,6$ ms (los ≈ 28 Gcélulas/s ya medidos). El costo de *host* por paso —`cudaLaunchKernel` ($\approx 15\ \mu\text{s}$) más los dos `cudaEventRecord` ($\approx 5\ \mu\text{s}$ c/u)— suma $\sim 25\text{--}30\ \mu\text{s}$: despreciable frente al kernel en grillas grandes (0,1 %) pero dominante en las pequeñas, lo que explica la rodilla de eficiencia de la Sección 5.2. La única transferencia, la subida inicial de 67 MB, tardó 8,0 ms, esto es 8,4 GB/s —por debajo del límite del enlace porque el buffer de *host* es *pageable* (un `std::vector`)—. Esto refina la Sección 3.5: transferir cada generación acotaría la GPU a ~ 8 Gcélulas/s (no ~ 12), reforzando la decisión de hacer el *ping-pong* en el *device*.

Nsight Compute (ncu).

Es el hallazgo que corrige el modelo. La Tabla 6 resume las métricas del kernel global y del de memoria compartida (bloque 128, $N = 4096$; el cuadro es idéntico en 8192). Tres lecturas:

- **El kernel global no está limitado por DRAM.** La L2 acierta el 88 % de las solicitudes, así que la DRAM solo mueve ≈ 2 bytes/celda y queda al ≈ 20 %. Cómputo y tubería de memoria están co-limitados al ~ 66 % de SOL, y el ~ 54 % de los ciclos se detiene esperando los datos de las cargas (*long scoreboard*): el kernel es **latency-bound sobre cargas servidas por caché**, no *bandwidth-bound* sobre DRAM.
- **Por qué pierde la memoria compartida.** Mueve los mismos ≈ 2 bytes/celda a DRAM (no gana nada ahí), pero su acierto en L1 se desploma del 67 % al 25 % (las cargas pasan ahora por la memoria compartida) y paga la carga cooperativa y la barrera. Acaba $\approx 1,9\times$ más lenta *pese* a una ocupación mayor (91 % vs. 82 %).
- **La ocupación no es el límite.** Ambos kernels superan el 80 % de ocupación alcanzada; más hilos no ayudarían. Esto cierra el porqué de la meseta del tamaño de bloque (Sección 4.3).

Tabla 6: Nsight Compute: kernel global *vs.* compartido (bloque 128, $N = 4096$).

Métrica	global	compartida
Rendimiento [Gcélulas/s]	28,0	14,8
Tráfico de DRAM [B/celda]	2,0	2,0
Uso de DRAM [% del pico]	19,7	10,4
Acierto de L2 [%]	87,9	87,4
Acierto de L1/TEX [%]	67,0	24,9
SOL cómputo / memoria [%]	66 / 66	62 / 45
Ocupación alcanzada [%]	81,7	90,7
<i>Stall</i> dominante	scoreboard (54 %)	scoreboard + barrera

El perfilamiento de OpenCL con `std::chrono` queda pendiente junto con ese backend.

6. Conclusiones

Se implementaron y compararon tres caminos del Juego de la Vida con la regla variante B36/S23: CPU secuencial, CPU paralela y CUDA (OpenCL queda pendiente). Los hallazgos principales:

- **Memoria compartida contraproducente.** Para un *stencil* de 1 byte, la versión con memoria compartida rinde $\approx 0,55\times$ la global: la caché L2 ya captura la reutilización y la barrera/halo no se amortizan. Esto valida el análisis *roofline*.
- **Tamaño de bloque.** Óptimo en 64–128 hilos, con curva cóncava por ocupación.
- **CPU.** Escala hasta los 6 núcleos físicos ($\approx 5,4\times$); el *hyperthreading* aporta poco y, por la sincronización con barrera, desbalancea en 8 y 10 hilos.
- **GPU.** ≈ 32 Gcélulas/s. El perfilamiento revela que *no* la limita el ancho de banda de DRAM (al $\approx 20\%$, pues la L2 sirve el 88 % de las solicitudes) sino la tubería L2/LSU y la latencia ($\sim 65\%$ de SOL); hay margen de mejora con más paralelismo de instrucción.
- **Speed-up.** Hasta $\approx 200\times$ sobre la mejor CPU paralela y $\approx 1200\times$ sobre la secuencial; la cota “limpia” de hardware es $\approx 6,8\times$, y la diferencia se explica por una CPU base no vectorizada.

Limitaciones y trabajo futuro. Falta el backend OpenCL y el perfilamiento con Nsight (Sección 5.3); la disposición de 1 byte por celda fija el techo de la GPU (empaquetar bits lo elevaría) y la CPU se beneficiaría de una reescritura vectorizada (AVX2) sin saltos.